Zadanie 5: programowanie genetyczne i regresja symboliczna

Paweł Knot, Jarosław Klima, Szymon Kowalski

Na 3.0

2. Zanotuj wzory trzech rozwiązań o najwyższej wartości `score` oraz rozwiązanie `best` dla następujących zestawów ustawień:
    1. `binary_operators=["+", "*"], unary_operators=["cos", "exp", "sin"], maxsize=20`,
    2. `binary_operators=["+", "*", "-", "^"], unary_operators=["cos", "exp", "sin", "log"], maxsize=30`, (dodaj ograniczenie dla argumentów operatora "^": https://astroautomata.com/PySR/v1.5.9/options.html#constraining-use-of-operators.
    3. `binary_operators=["+", "*", "-", "^"], unary_operators=["exp", "sin"], maxsize=15`.
3. Powtórz eksperymenty z zadania na 3.0 po dodaniu szumu do próbek z funkcji f (rozkład normalny o średniej 0 i odchyleniu standardowym 0.5)

Na 4.0

Do realizacji:

1. Punkty z zadania na 3.0.
2. Dodaj do porównania dopasowanie oparte o próbki losowane w szerszym zakresie (między -15 a 15) oraz wyższy poziom szumu (odchylenie standardowe równe 2 oraz 5).

Na 5.0

Do realizacji:

1. Punkty z zadania na 4.0.
2. Zamień funkcję f na $f(x) = 2.2\sin(x_0 + 2x_1) - x_5^2 - p(\lfloor x_0 \rfloor)$ gdzie $p(i)$ oznacza i-tą liczbę pierwszą. Uwzględnij `p` jako dodatkowy operator unarny analogicznie do przykładu "Julia packages and types" z notatnika `pysr_demo.ipynb`. Powtórz eksperymenty opisane w zadaniach na 3.0 i 4.0.

Zadanie 1 (Na 3.0)

Ekspeymenty bez sumu

```
import pysr
import sympy
import numpy as np
from matplotlib import pyplot as plt
from pysr import PySRRegressor
from sklearn.model_selection import train_test_split

np.random.seed(0)
X = np.random.uniform(-5, 5, size=(200, 6))
y = 2.2 * np.sin(X[:, 0]+ 2*X[:,1]) - X[:,5]**2 - 3
```

```
Detected IPython. Loading juliacall extension. See https://juliapy.github.io/PythonCall.jl/st
```

```
default_pysr_params = dict(
    populations=30,
    model_selection="best",
)
model = PySRRegressor(
    niterations=100,
    binary_operators=["+", "*"],
    unary_operators=["cos", "exp", "sin"],
    maxsize=20,
    verbosity=False,
    **default_pysr_params,
)
```

```
model.fit(X, y)
eqs = model.equations_
top_3_scores = eqs.sort_values("score",ascending=False).head(3)
for i, row in top_3_scores.iterrows():
    print(f"Miejsce {i+1} (Score: {row['score']:.4f}):")
    print(f"Wzór: {row['sympy_format']}")
    print("-"*30)
model.sympy()
```

```
C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Use
  warnings.warn(
```

```
Miejsce 11 (Score: 13.7186):
Wzór: x5*x5*(-1.0) + sin(x0 + x1 + x1)*2.2 - 3.0
------------------------------
Miejsce 4 (Score: 2.0312):
Wzór: x5*x5*(-1.1931673)
------------------------------
Miejsce 10 (Score: 1.1323):
Wzór: x5*x5*(-0.98993224) + sin(x0 + x1 + x1) - 3.088995
------------------------------
```

$$x_5 x_5 \left(-1.0\right) + \sin\left(x_0 + x_1 + x_1\right) 2.2 - 3.0$$

```
model = PySRRegressor(
    niterations=100,
    binary_operators=[
        "+", "*", "-",
        "my_pow(x,y) = (x > 0) ? x^y : convert(typeof(x), NaN)"
    ],
    extra_sympy_mappings={
        "my_pow": lambda x, y: x**y
    },
    unary_operators=["cos", "exp", "sin", "log"],
    maxsize=30,
    verbosity=False,
    constraints={'my_pow': (-1,1)},
    **default_pysr_params,
)
model.fit(X, y)
eqs = model.equations_
```

```
top_3_scores = eqs.sort_values("score",ascending=False).head(3)
for i, row in top_3_scores.iterrows():
    print(f"Miejsce {i+1} (Score: {row['score']:.4f}):")
    print(f"Wzór: {row['sympy_format']}")
    print("-"*30)
model.sympy()
```

```
C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Use
  warnings.warn(

Miejsce 11 (Score: 13.0110):
Wzór: x5*x5*(-0.99999994) + sin(x0 + x1*1.9999998)*2.2000008 - 3.0000007
------------------------------
Miejsce 4 (Score: 3.1427):
Wzór: -x5*x5 - 3.0128005
------------------------------
Miejsce 10 (Score: 0.7885):
Wzór: x5*x5*(-0.9897953) + sin(x0 + x1*2.0043526) - 3.0895226
------------------------------
```

$$x_5 x_5 \left(-0.99999994\right) + \sin\left(x_0 + x_1 \cdot 1.9999998\right) 2.2000008 - 3.0000007$$

```
model = PySRRegressor(
    niterations=100,
    binary_operators=["+", "*", "-","my_pow(x,y) = (x > 0) ? x^y : convert(typeof(x), NaN)"]
    extra_sympy_mappings={
        "my_pow": lambda x, y: x**y
    },
    unary_operators=["exp", "sin"],
    maxsize=15,
    verbosity=False,
    constraints={'my_pow': (-1,1)},
    **default_pysr_params,
)
model.fit(X, y)
eqs = model.equations_
top_3_scores = eqs.sort_values("score",ascending=False).head(3)
for i, row in top_3_scores.iterrows():
    print(f"Miejsce {i+1} (Score: {row['score']:.4f}):")
    print(f"Wzór: {row['sympy_format']}")
    print("-"*30)
model.sympy()
```

```
C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Use
  warnings.warn(

Miejsce 4 (Score: 3.2011):
Wzór: -x5*x5 - 3.0129473
------------------------------
Miejsce 9 (Score: 1.1329):
Wzór: -x5*x5 + sin(x0 + x1 + x1) - 3.0069585
------------------------------
Miejsce 8 (Score: 0.0247):
Wzór: -x5*x5 + sin(exp(x1 + 1.0187684)) - 3.2134345
------------------------------
```

$$-x_5 x_5 + \sin\left(x_0 + x_1 + x_1\right) - 3.0069585$$

Eksperymenty z szumem

```python
np.random.seed(0)
X = np.random.uniform(-5, 5, size=(200, 6))
y = 2.2 * np.sin(X[:, 0]+ 2*X[:,1]) - X[:,5]**2 - 3
noise = 0.5 * np.random.randn(200)
y_noised = y + noise
```

```python
default_pysr_params = dict(
    populations=30,
    model_selection="best",
)
model = PySRRegressor(
    niterations=100,
    binary_operators=["+", "*"],
    unary_operators=["cos", "exp", "sin"],
    maxsize=20,
    verbosity=False,
    **default_pysr_params,
)
model.fit(X, y_noised)
eqs = model.equations_
top_3_scores = eqs.sort_values("score",ascending=False).head(3)
for i, row in top_3_scores.iterrows():
    print(f"Miejsce {i+1} (Score: {row['score']:.4f}):")
```

```
    print(f"Wzór: {row['sympy_format']}")
    print("-"*30)
model.sympy()
```

```
C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Use
  warnings.warn(
```

```
Miejsce 4 (Score: 2.0183):
Wzór: x5*x5*(-1.1879786)
------------------------------
Miejsce 12 (Score: 1.3250):
Wzór: x5*(-0.99840987)*x5 + sin(x0 + x1 + x1)*2.2154784 - 2.9473782
------------------------------
Miejsce 10 (Score: 0.9516):
Wzór: x5*x5*(-0.98821336) + sin(x0 + x1 + x1) - 3.0375047
------------------------------
```

$$x_5 \left(-0.99840987\right) x_5 + \sin\left(x_0 + x_1 + x_1\right) 2.2154784 - 2.9473782$$

```
model = PySRRegressor(
    niterations=100,
    binary_operators=["+", "*", "-","my_pow(x,y) = (x > 0) ? x^y : convert(typeof(x), NaN)"]
    extra_sympy_mappings={
        "my_pow": lambda x, y: x**y
    },
    unary_operators=["cos", "exp", "sin", "log"],
    maxsize=30,
    verbosity=False,
    constraints={'pow': (-1,1)},
    **default_pysr_params,
)
model.fit(X, y_noised)
eqs = model.equations_
top_3_scores = eqs.sort_values("score",ascending=False).head(3)
for i, row in top_3_scores.iterrows():
    print(f"Miejsce {i+1} (Score: {row['score']:.4f}):")
    print(f"Wzór: {row['sympy_format']}")
    print("-"*30)
model.sympy()
```

```
C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Use
  warnings.warn(


Miejsce 4 (Score: 3.0386):
Wzór: -x5*x5 - 2.9473784
------------------------------
Miejsce 13 (Score: 1.3140):
Wzór: -(x5*x5 - (-2.216142)*cos(x0 + x1 + x1 - 10.983329)) - 2.935281
------------------------------
Miejsce 11 (Score: 0.8262):
Wzór: -x5*x5 - cos(x0 + x1 + x1 - 1*(-1.6053224)) - 2.942641
------------------------------
```

$$-(x_5 x_5 - \cos(x_0 + x_1 + x_1 - 10.983329)\,(-2.216142)) - 2.935281$$

```python
model = PySRRegressor(
    niterations=100,
    binary_operators=["+", "*", "-","my_pow(x,y) = (x > 0) ? x^y : convert(typeof(x), NaN)"]
    extra_sympy_mappings={
        "my_pow": lambda x, y: x**y
    },
    unary_operators=["exp", "sin"],
    maxsize=15,
    verbosity=False,
    constraints={'pow': (-1,1)},
    **default_pysr_params,
)
model.fit(X, y_noised)
eqs = model.equations_
top_3_scores = eqs.sort_values("score",ascending=False).head(3)
for i, row in top_3_scores.iterrows():
    print(f"Miejsce {i+1} (Score: {row['score']:.4f}):")
    print(f"Wzór: {row['sympy_format']}")
    print("-"*30)
model.sympy()
```

```
C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Use
  warnings.warn(


Miejsce 4 (Score: 3.0967):
Wzór: -x5*x5 - 2.9473786
```

```
------------------------------
Miejsce 9 (Score: 0.9541):
Wzór: -x5*x5 + sin(x0 + x1 + x1) - 2.941498
------------------------------
Miejsce 10 (Score: 0.1043):
Wzór: -x5*x5 + exp(sin(x0 + x1 + x1)) - 4.1842823
------------------------------
```

$-x_5 x_5 + \sin(x_0 + x_1 + x_1) - 2.941498$

Zadanie na 4.0

```python
np.random.seed(0)
X_wide = np.random.uniform(-15, 15, size=(200, 6))
y_wide = 2.2 * np.sin(X_wide[:, 0] + 2 * X_wide[:, 1]) - X_wide[:, 5]**2 - 3

configs = [
    dict(
        binary_operators=["+", "*"],
        unary_operators=["cos", "exp", "sin"],
        maxsize=20,
    ),
    dict(
        binary_operators=["+", "*", "-", "my_pow(x,y) = (x > 0) ? x^y : convert(typeof(x), Na
        unary_operators=["cos", "exp", "sin", "log"],
        maxsize=30,
        constraints={"my_pow": (-1, 1)},
        extra_sympy_mappings={"my_pow": lambda x, y: x**y},
    ),
    dict(
        binary_operators=["+", "*", "-", "my_pow(x,y) = (x > 0) ? x^y : convert(typeof(x), Na
        unary_operators=["exp", "sin"],
        maxsize=15,
        constraints={"my_pow": (-1, 1)},
        extra_sympy_mappings={"my_pow": lambda x, y: x**y},
    ),
]

for noise_std in [0.0, 2.0, 5.0]:
    print(f"\nSzum std = {noise_std}")
```

```
    y_train = y_wide + noise_std * np.random.randn(len(y_wide))
    for config_index, config in enumerate(configs, start=1):
        model = PySRRegressor(
            niterations=100,
            populations=30,
            model_selection="best",
            verbosity=False,
            **config,
        )
        model.fit(X_wide, y_train)
        top_3_scores = model.equations_.sort_values("score", ascending=False).head(3)
        print(f"Konfiguracja {config_index}:")
        for rank, (_, row) in enumerate(top_3_scores.iterrows(), start=1):
            print(f"  {rank}. score={row['score']:.4f}, wzór={row['sympy_format']}")
        print(f"  best: {model.sympy()}")
        print("-" * 30)
```

```
Szum std = 0.0

C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Use
  warnings.warn(

Konfiguracja 1:
  1. score=3.2730, wzór=x5*x5*(-1.0218029)
  2. score=0.5546, wzór=x5*x5*(-0.9992922) - 3.028511
  3. score=0.0539, wzór=x5*x5*(-0.99981) + sin(exp(x2)*0.32728404) - 2.99932
  best: x5*x5*(-0.9992922) - 3.028511
------------------------------

C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Use
  warnings.warn(

Konfiguracja 2:
  1. score=23.3026, wzór=-x5*x5 + cos(x0 + x1 + x1 - 1.5707972)*2.2000008 - 2.9999998
  2. score=3.8271, wzór=-x5*x5 - 2.9766607
  3. score=0.5798, wzór=-(x5*x5 + cos(x0 + x1 + x1 + 1.5485848)) - 2.985967
  best: -x5*x5 + cos(x0 + x1 + x1 - 1.5707972)*2.2000008 - 2.9999998
------------------------------
```

```
C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Use
  warnings.warn(

Konfiguracja 3:
  1. score=3.8271, wzór=-x5*x5 - 2.9766607
  2. score=0.0318, wzór=-x5*x5 + sin(x0*(-1.4144974))*(-0.526893) - 2.984826
  3. score=0.0167, wzór=x2 - 76.74551
  best: -x5*x5 - 2.9766607
------------------------------

Szum std = 2.0

C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Use
  warnings.warn(

Konfiguracja 1:
  1. score=3.0473, wzór=x5*x5*(-1.0218347)
  2. score=0.3146, wzór=x5*x5*(-0.998307) - 3.1665735
  3. score=0.0688, wzór=(x5*x5 + sin(x0*(-1.4328047)))*(-0.99811155) - 3.2064438
  best: x5*x5*(-0.998307) - 3.1665735
------------------------------

C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Use
  warnings.warn(

Konfiguracja 2:
  1. score=3.3607, wzór=-x5*x5 - 3.0429764
  2. score=0.1942, wzór=-x5*x5 + sin(x0 + x1 + x1) - 3.0534074
  3. score=0.0601, wzór=-x5*x5 + sin(x4*4.7092204) - 3.1032746
  best: -x5*x5 + sin(x0 + x1 + x1) - 3.0534074
------------------------------

C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Use
  warnings.warn(

Konfiguracja 3:
  1. score=3.3607, wzór=-x5*x5 - 3.0431
  2. score=0.1848, wzór=-x5*x5 + sin(x0 + x1 + x1) - 3.053447
  3. score=0.0652, wzór=-x5*x5 + sin(x0*1.4332799) - 3.068958
  best: -x5*x5 - 3.0431
```

```
------------------------------

Szum std = 5.0

C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Use
  warnings.warn(

Konfiguracja 1:
  1. score=2.5102, wzór=x5*x5*(-1.021107)
  2. score=0.1538, wzór=x5*(-0.99095625)*x5 - 4.0558853
  3. score=0.0526, wzór=x5*x5*(-0.9909196) + sin(x2) - 4.0528164
  best: x5*(-0.99095625)*x5 - 4.0558853
------------------------------

C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Use
  warnings.warn(

Konfiguracja 2:
  1. score=2.6553, wzór=-x5*x5 - 3.3932395
  2. score=0.0384, wzór=-x5*x5 + sin(x2) - 3.3870008
  3. score=0.0210, wzór=-x5*x5 - (3.4634016 - cos(x5*(-15.95287))) + sin(x2)
  best: -x5*x5 - 3.3932395
------------------------------

C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Use
  warnings.warn(

Konfiguracja 3:
  1. score=2.6553, wzór=-x5*x5 - 3.3932152
  2. score=0.0387, wzór=-x5*x5 + sin(x2) - 3.38729
  3. score=0.0182, wzór=x5*x5*(-0.9900652) + sin(x2) - sin(x5) - 4.174001
  best: -x5*x5 - 3.3932152
------------------------------
```

Zadanie na 5.0

```
from pysr import jl

jl.seval("""
import Pkg
Pkg.add("Primes")
""")

jl.seval("using Primes: prime")

jl.seval("""
function p(x::T) where T
    if isnan(x) || isinf(x) || x < 1.0 || x >= 1000.0
        return T(NaN)
    end
    floor_val = floor(Int, x)
    return T(prime(floor_val))
end
""")
```

```
   Resolving package versions...
  No Changes to `C:\Users\szyro\.julia\environments\pyjuliapkg\Project.toml`
  No Changes to `C:\Users\szyro\.julia\environments\pyjuliapkg\Manifest.toml`
```

```
p(1.9) = 2.0 (oczekiwane: 2.0, bo prime(floor(1.9))=prime(1)=2)
p(3.0) = 5.0 (oczekiwane: 5.0, bo prime(3)=5)
p(0.9) = nan (oczekiwane: NaN, bo floor(0.9)=0 < 1)
p(-1.0) = nan (oczekiwane: NaN, bo floor(-1.0)=-1 < 1)
```

```
class sympy_p(sympy.Function):
    pass


def compute_y_prime(X):
    """Oblicza f(x) = 2.2*sin(x0 + 2*x1) - x5^2 - p(floor(x0))."""
    base = 2.2 * np.sin(X[:, 0] + 2 * X[:, 1]) - X[:, 5]**2
    p_vals = np.array([float(jl.p(x0)) for x0 in X[:, 0]])
    return base - p_vals
```

```
def make_dataset_prime(lo, hi, n=200, noise_std=0.0, seed=0):
    """Generuje n próbek z zakresu [lo, hi]."""
    rng = np.random.RandomState(seed)
    collected_X, collected_y = [], []

    while sum(len(a) for a in collected_X) < n:
        X_batch = rng.uniform(lo, hi, size=(n * 3, 6))
        y_batch = compute_y_prime(X_batch)
        valid = ~np.isnan(y_batch)
        collected_X.append(X_batch[valid])
        collected_y.append(y_batch[valid])

    X_all = np.vstack(collected_X)[:n]
    y_all = np.concatenate(collected_y)[:n]

    if noise_std > 0:
        y_all = y_all + noise_std * rng.randn(n)

    return X_all, y_all


configs_prime = [
    dict(
        binary_operators=["+", "*"],
        unary_operators=["cos", "exp", "sin", "p"],
        maxsize=20,
        extra_sympy_mappings={"p": sympy_p},
    ),
    dict(
        binary_operators=["+", "*", "-", "my_pow(x,y) = (x > 0) ? x^y : convert(typeof(x), Na
        unary_operators=["cos", "exp", "sin", "log", "p"],
        maxsize=30,
        constraints={"my_pow": (-1, 1)},
        extra_sympy_mappings={"my_pow": lambda x, y: x**y, "p": sympy_p},
    ),
    dict(
        binary_operators=["+", "*", "-", "my_pow(x,y) = (x > 0) ? x^y : convert(typeof(x), Na
        unary_operators=["exp", "sin", "p"],
        maxsize=15,
        constraints={"my_pow": (-1, 1)},
        extra_sympy_mappings={"my_pow": lambda x, y: x**y, "p": sympy_p},
    ),
```

```
]
```

```
for noise_std in [0.0, 0.5]:
    X_p, y_p = make_dataset_prime(-5, 5, n=200, noise_std=noise_std, seed=0)
    print(f"\n{'='*50}")
    print(f"Zakres [-5, 5], szum std = {noise_std}")
    print(f"{'='*50}")
    for config_index, config in enumerate(configs_prime, start=1):
        cfg = config.copy()
        model = PySRRegressor(
            niterations=100,
            populations=30,
            model_selection="best",
            verbosity=False,
            **cfg,
        )
        model.fit(X_p, y_p)
        top_3_scores = model.equations_.sort_values("score", ascending=False).head(3)
        print(f"Konfiguracja {config_index}:")
        for rank, (_, row) in enumerate(top_3_scores.iterrows(), start=1):
            print(f"  {rank}. score={row['score']:.4f}, wzór={row['sympy_format']}")
        print(f"  best: {model.sympy()}")
        print("-" * 30)
```

```
==================================================
Zakres [-5, 5], szum std = 0.0
==================================================

C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Use
  warnings.warn(

Konfiguracja 1:
  1. score=1.1984, wzór=x5*x5*(-1.2732484)
  2. score=0.7615, wzór=x0*(-1.4877833) + x5*(-0.9808544)*x5 + sin(x0 + x1*2.015316)
  3. score=0.7354, wzór=(x0 + x5*x5)*(-1.0759963)
  best: x0*(-1.4833608) + x5*x5*(-0.9874065) + sin(x0 + x1*2.0116534)*2.1473374
------------------------------

C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Use
  warnings.warn(
```

```
Konfiguracja 2:
  1. score=13.7295, wzór=-(x5*x5 + sympy_p(x0)) - 2.2000003*sin(-x0 + x1*(-
2.0)) - 2.3416852e-7
  2. score=2.1350, wzór=-x5*x5 - 4.348845
  3. score=0.6797, wzór=(x5*x5 + sympy_p(x0))*(-0.9954726) - sin(-x0 + x1*(-
2.004592))
  best: -(x5*x5 + sympy_p(x0)) - 2.2000003*sin(-x0 + x1*(-2.0)) - 2.3416852e-
7
------------------------------


C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Use
  warnings.warn(

Konfiguracja 3:
  1. score=1.0893, wzór=-x5*x5 - 4.3489676
  2. score=0.4324, wzór=x0*(-1.4369632) - x5*x5
  3. score=0.4159, wzór=-x0 - x5*x5 + sin(x0 - (-1.993454)*x1) - 1.3085734
  best: -x0 - x5*x5 + sin(x0 - (-1.993454)*x1) - 1.3085734
------------------------------

==================================================
Zakres [-5, 5], szum std = 0.5
==================================================


C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Use
  warnings.warn(

Konfiguracja 1:
  1. score=1.1931, wzór=x5*x5*(-1.2765152)
  2. score=0.6991, wzór=(x0 + x5*x5)*(-1.078314)
  3. score=0.3591, wzór=(x5*x5 + sympy_p(x0))*(-0.99427044)
  best: (x5*x5 + sympy_p(x0))*(-0.99427044)
------------------------------


C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Use
  warnings.warn(

Konfiguracja 2:
  1. score=2.1174, wzór=-x5*x5 - 4.379534
  2. score=0.5695, wzór=-x5*x5 - 1.7011225*(x0 - cos(x0 + (x1 - 0.7870843)*2.0317633)) + 0.79
  3. score=0.3786, wzór=x0*(-1.4385372) - x5*x5
```

```
  best: -x0 - x5*x5 + sin(x0 + x1*2.0202785) + sin(x0 + x1*2.0202785) + sin(exp(sympy_p(x0) -
------------------------------

C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Use
  warnings.warn(

Konfiguracja 3:
  1. score=1.0819, wzór=-x5*x5 - 4.3791738
  2. score=0.8051, wzór=-(x5*x5 + sympy_p(x0)) + sin(x0 + x1 + x1)*2.2728498
  3. score=0.4213, wzór=-(x5*x5 + sympy_p(x0)) + sin(x0 + x1 + x1)
  best: -(x5*x5 + sympy_p(x0)) + sin(x0 + x1 + x1)*2.2728498
------------------------------
```

```python
for noise_std in [0.0, 2.0, 5.0]:
    X_p, y_p = make_dataset_prime(-15, 15, n=200, noise_std=noise_std, seed=0)
    print(f"\n{'='*50}")
    print(f"Zakres [-15, 15], szum std = {noise_std}")
    print(f"{'='*50}")
    for config_index, config in enumerate(configs_prime, start=1):
        cfg = config.copy()
        model = PySRRegressor(
            niterations=100,
            populations=30,
            model_selection="best",
            verbosity=False,
            **cfg,
        )
        model.fit(X_p, y_p)
        top_3_scores = model.equations_.sort_values("score", ascending=False).head(3)
        print(f"Konfiguracja {config_index}:")
        for rank, (_, row) in enumerate(top_3_scores.iterrows(), start=1):
            print(f"  {rank}. score={row['score']:.4f}, wzór={row['sympy_format']}")
        print(f"  best: {model.sympy()}")
        print("-" * 30)
```

```
==================================================
Zakres [-15, 15], szum std = 0.0
==================================================

C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Use
  warnings.warn(
```

```
Konfiguracja 1:
  1. score=4.3913, wzór=(x5*x5 + sympy_p(x0))*(-0.9997017)
  2. score=1.1516, wzór=(x5*x5 + sympy_p(x0))*(-0.9998341) + sin(x0 + x1 + x1)
  3. score=1.1516, wzór=x5*x5*(-1.1508262)
  best: (x5*x5 + sympy_p(x0) + cos(exp(sin(x0 + x1 + x1)))*2.0151107)*(-
0.9962152)
------------------------------


C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Use
  warnings.warn(


Konfiguracja 2:
  1. score=11.7675, wzór=-x5*x5 - sympy_p(x0) + sin(x0 + x1 + x1)*2.2000015
  2. score=1.9931, wzór=x5*(0.012355536 - x5) - sympy_p(x0)
  3. score=1.5236, wzór=-x5*x5 - 21.533724
  best: -x5*x5 - sympy_p(x0) + sin(x0 + x1 + x1)*2.2000015
------------------------------


C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Use
  warnings.warn(


Konfiguracja 3:
  1. score=1.9931, wzór=x5*(0.012355524 - x5) - sympy_p(x0)
  2. score=1.5236, wzór=-x5*x5 - 21.53443
  3. score=1.2764, wzór=x0*(-2.7270088) - x5*x5
  best: -x5*x5 - sympy_p(x0) + sin(x0 + x1 + x1)
------------------------------


==================================================
Zakres [-15, 15], szum std = 2.0
==================================================


C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Use
  warnings.warn(


Konfiguracja 1:
  1. score=3.5404, wzór=(x5*x5 + sympy_p(x0))*(-1.0014504)
  2. score=1.1509, wzór=x5*x5*(-1.1528928)
  3. score=0.4468, wzór=(x0 + x5*x5)*(-1.095538)
  best: (x5*x5 + sympy_p(x0))*(-1.0014504)
------------------------------
```

```
C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Use
  warnings.warn(

Konfiguracja 2:
  1. score=3.0458, wzór=x5*x5*(-1.0016773) - sympy_p(x0)
  2. score=1.5160, wzór=-x5*x5 - 21.657587
  3. score=0.3246, wzór=-x0 - x5*x5 - 13.234107
  best: -x5*x5 - sympy_p(x0) + sin(x0 + x1 + x1)
------------------------------

C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Use
  warnings.warn(

Konfiguracja 3:
  1. score=1.5160, wzór=-x5*x5 - 21.657587
  2. score=1.2708, wzór=x5*x5*(-1.0016752) - sympy_p(x0)
  3. score=1.2120, wzór=x0*(-2.7418737) - x5*x5
  best: x5*x5*(-1.0016752) - sympy_p(x0)
------------------------------

==================================================
Zakres [-15, 15], szum std = 5.0
==================================================

C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Use
  warnings.warn(

Konfiguracja 1:
  1. score=2.1887, wzór=(x5*x5 + sympy_p(x0))*(-1.0040649)
  2. score=1.1376, wzór=x5*x5*(-1.1559776)
  3. score=0.4218, wzór=(x0 + x5*x5)*(-1.0984504)
  best: (x5*x5 + sympy_p(x0))*(-1.0040649)
------------------------------

C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Use
  warnings.warn(

Konfiguracja 2:
  1. score=1.4785, wzór=-x5*x5 - 21.842121
  2. score=0.9414, wzór=x0*(-2.7642477) - x5*x5
```

```
  3. score=0.4519, wzór=-x5*x5 - sympy_p(x0) - 0.29729593
  best: -x5*x5 - sympy_p(x0) - 0.29729593
------------------------------


C:\Users\szyro\AppData\Local\Programs\Python\Python313\Lib\site-packages\pysr\sr.py:2811: Us
  warnings.warn(


Konfiguracja 3:
  1. score=1.4785, wzór=-x5*x5 - 21.842646
  2. score=0.9414, wzór=x0*(-2.7641513) - x5*x5
  3. score=0.4519, wzór=-(x5*x5 + sympy_p(x0)) - 0.29755434
  best: -(x5*x5 + sympy_p(x0)) - 0.29755434
------------------------------
```